Project Title

Word-Level Plagiarism Checker for Short Documents

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
Surabhi Udayasri (2520090003)
Bushetty Akshitha (2520090014)

Team Number: 06

Section: 12

Under the guidance of

Ms. K. Chandusha

Assistant Professor, CS&IT



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



DECLARATION

The Mini Project Report entitled "**Word-Level Plagiarism Checker for Short Documents"** is a record of Bonafide work of **Surabhi Udayasri – 2520090003, Bushetty Akshitha – 2520090014** submitted in partial fulfillment for the award of B. Tech in Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Surabhi Udayasri – 2520090003

Bushetty Akshitha – 2520090014

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled "**Word-Level Plagiarism Checker for Short Documents"** is a bonafide work done and submitted by **Surabhi Udayasri – 2520090003, Bushetty Akshitha – 2520090014** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Information Technology, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, Ms. K. Chandusha, Assistant Professor, Department of CS&IT, which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Khaja Shareef (CSIT)**, Head of the *Department of Computer Science and Information Technology* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Surabhi Udayasri – 2520090003

Bushetty Akshitha – 2520090014

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	10
5.	Hardware and Software requirements	11
6.	Implementation	12
7.	Results & Discussion	15
8.	Conclusion & Future Scope	17
9.	References	18

1.ABSTRACT

Plagiarism, the act of copying another person's work without giving proper credit, is a persistent problem in schools, colleges and publishing. Manual review of assignments, reports and essays is slow and often inconsistent, particularly when a submission has to be compared against many reference documents.

This project presents a Word-Level Plagiarism Checker for short documents. The system takes a submitted document and a set of reference documents as input, cleans and tokenizes the text into words, and indexes the reference words in a hash table. Every word of the submission is then searched against this table, and consecutive three-word phrases are checked to detect copied sentences. The matches are used to compute an overall similarity percentage and to report the source document from which the matched content came.

The solution is built entirely on core Data Structures and Algorithms: arrays and strings for storing and processing text, hashing for near constant-time lookup, and linear and hash-based searching for matching. This keeps the tool lightweight, fast and easy to use for teachers, editors and publishers. In the test runs, a copied submission scored 100% similarity with the correct sources identified, whereas an original submission scored only 15%.

Team Name: Team 06 (Section 12)

Team Logo (if any): Not applicable

Team Members:

1. Surabhi Udayasri (2520090003)
2. Bushetty Akshitha (2520090014)

2. INTRODUCTION

Academic and publishing environments generate large volumes of written work, which makes it difficult to verify the originality of every document by hand. Plagiarism, which means using another person's words or ideas without proper acknowledgement, harms the fairness of evaluation and the credibility of published work. A lightweight, algorithmic checker can flag copied content quickly, before manual review even begins.

The Problem: Manually comparing a submission against several reference documents is time-consuming and error-prone, and the effort grows rapidly as the document collection grows.

The Goal: The goal of this project is to automatically detect word-level and phrase-level overlap between a submitted document and a set of reference documents, and to express that overlap as a similarity score together with the matching source.

The Approach: The text is first tokenized into words using arrays and strings. The words of the reference documents are then indexed in a hash table, which gives O(1) average lookup time, and each word of the submitted document is searched in that table. The percentage of matched words gives the similarity score.

The Users: The tool is intended for teachers who check assignments, editors who verify manuscripts and publishers who screen submissions for originality.

The rest of this report is organized as follows: Section 3 reviews related work, Section 4 states the problem and objectives, Section 5 lists the hardware and software requirements, Section 6 presents the implementation, Section 7 discusses the results and Section 8 gives the conclusion and future scope.

3. LITERATURE SURVEY

The following existing systems and techniques for plagiarism and similarity detection were studied before designing this project.

[1] Turnitin and commercial plagiarism checkers: These services compare a submitted document against a very large database of web pages, journals and previously submitted papers using corpus matching and text fingerprinting. They are accurate for large-scale screening, but they are closed-source, depend on heavy server infrastructure and are costly for small institutions or individual teachers.

[2] MOSS (Measure of Software Similarity): MOSS, developed at Stanford, detects similarity between programs using a document-fingerprinting method with string tiling. It works very well for source code, where identifiers and structure matter, but it is built for programming languages and is not suitable for comparing natural-language text word by word.

[3] Cosine similarity and TF-IDF methods: Each document is converted into a vector of weighted term frequencies and the angle between the vectors is used as the similarity measure. The method is simple and widely used, but it treats the text as a bag of words, so word order is ignored and a copied sentence cannot be located.

[4] N-gram fingerprinting: Overlapping sequences of n consecutive words are hashed and the hash values of two documents are compared. This catches copied phrases and sentences well, but the result depends on the chosen n; a large n misses short copied fragments and a small n produces many false matches.

[5] Rabin–Karp style string matching: A rolling hash is used to compare a pattern against every window of a text in nearly linear time. It is very efficient for finding exact substrings, but it fails when the copied text is paraphrased or when a few words are changed.

Comparison of existing approaches

System / Approach	Technique Used	Limitation
Turnitin / commercial checkers	Large-scale corpus matching, fingerprinting	Closed-source, heavy infrastructure, costly for small use
MOSS (Measure of Software Similarity)	String tiling for code similarity	Built for source code, not natural-language text

System / Approach	Technique Used	Limitation
Cosine similarity / TF-IDF methods	Vector-space document comparison	Treats text as a bag of words; ignores word order
N-gram fingerprinting	Hashing overlapping word sequences	Sensitive to n-gram size; can miss single-word matches
Rabin–Karp style string matching	Rolling hash for substring search	Efficient for exact matches, weaker on paraphrased text

Gap identified: most existing tools are either too heavy for short documents or optimized for code, not for word-level natural-language comparison. This motivates a lightweight, hashing-based word-level checker.

4. Problem Statement and Objectives

Problem Statement:

To build a simple and fast word-level plagiarism checker that compares a submitted short document with reference documents and reports a similarity percentage and the matched source.

Problem Description:

Manual comparison of assignments and manuscripts with reference material is slow and inconsistent, and existing tools are heavy, costly or meant for source code. A lightweight DSA-based method is needed for short natural-language documents.

Objectives:

The objectives of the project are:

- To load a submitted document and a collection of reference documents and clean the text (lowercase, punctuation removal, tokenization) using arrays and strings.
- To index the reference words in a hash table so that each word can be looked up in O(1) average time.
- To search every word and every three-word phrase of the submitted document against the hash table to detect copied content.
- To compute the similarity percentage as matched words over total words.
- To report the source document from which the matched content came, so that the result can be verified.
- To demonstrate the practical use of arrays, strings, hashing and searching in a real-world application.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Processor: Intel Core i3 (or equivalent) or higher
- RAM: 4 GB minimum (8 GB recommended)
- Storage: 500 MB of free disk space
- Display, keyboard and mouse

Software Requirements:

- **Operating System:** Windows 10/11, Linux, or macOS
- **Programming Language:** Python 3.x (or Java, as per implementation choice)
- **IDE / Editor:** VS Code / PyCharm / Jupyter Notebook
- **Core Libraries:** Built-in string, collections (hash table/dict) and file I/O modules
- **Version Control:** Git & GitHub for source management
- **Documentation Tool:** MS Word / Google Docs for reports

6. IMPLEMENTATION

The project is implemented in Python using only built-in modules (re, os, sys, collections). It consists of four modules, each corresponding to a step of the proposed methodology. The complete source code is given below.

Module 1: Input and Preprocessing (Arrays and Strings)

This module loads a document, converts it to lowercase, removes punctuation and splits it into an array of words.

```
import re, os, sys
from collections import defaultdict

def load_document(path):
    with open(path, "r", encoding="utf-8") as f:
        return f.read()

def preprocess(text):
    """Lowercase, strip punctuation, tokenize into an array of words."""
    text = text.lower()
    text = re.sub(r"[^a-z0-9\s]", " ", text)
    return text.split()
```

Module 2: Indexing (Hashing)

This module builds hash tables (Python dictionaries) that map every reference word, and every three-word phrase, to the set of source documents containing it.

```
def build_index(reference_docs):
    """Hash table: word -> set of source document names. O(1) average lookup."""
    index = defaultdict(set)
    for name, words in reference_docs.items():
        for w in words:
            index[w].add(name)
    return index

def build_phrase_index(reference_docs, n=3):
    """Hash table of n-word phrases (shingles) -> source names."""
    phrases = defaultdict(set)
    for name, words in reference_docs.items():
        for i in range(len(words) - n + 1):
            phrases[" ".join(words[i:i + n])].add(name)
    return phrases
```

Module 3: Word-by-Word Matching (Searching)

Each word of the submitted document is scanned linearly and looked up in the hash table; phrase matching uses the same approach on three-word windows.

```
def match_words(words, index):
    matched, per_source = [], defaultdict(int)
    for w in words:                     # linear scan of submitted document
        if w in index:                  # hash lookup
            matched.append(w)
            for src in index[w]:
                per_source[src] += 1
    return matched, per_source

def match_phrases(words, phrase_index, n=3):
    hits = []
    for i in range(len(words) - n + 1):
        p = " ".join(words[i:i + n])
        if p in phrase_index:
            hits.append((p, sorted(phrase_index[p])))
    return hits
```

Module 4: Similarity Calculation and Report Generation

This module computes the similarity percentage, prints the source-wise match counts and the copied phrases, and gives a verdict. The main block ties all four modules together.

```
def similarity(matched, total):
    return 0.0 if total == 0 else round(100.0 * len(matched) / total, 2)

def report(sub_name, words, matched, per_source, phrase_hits):
    print("=" * 56)
    print(" WORD-LEVEL PLAGIARISM CHECKER - REPORT")
    print("=" * 56)
    print(f"Submitted document : {sub_name}")
    print(f"Total words        : {len(words)}")
    print(f"Matched words      : {len(matched)}")
    print(f"Similarity score   : {similarity(matched, len(words))} %")
    print("-" * 56)
    print("Source-wise word matches:")
    for src, c in sorted(per_source.items(), key=lambda x: -x[1]):
        print(f"  {src:<14} -> {c} words ({round(100*c/len(words),2)} %)")
    print("-" * 56)
    print(f"Copied 3-word phrases found: {len(phrase_hits)}")
    for p, srcs in phrase_hits[:5]:
        print(f"  \"{p}\"  <- {', '.join(srcs)}")
    v = similarity(matched, len(words))
    verdict = "HIGH - likely plagiarised" if v >= 60 else "MODERATE - review needed" if
v >= 30 else "LOW - mostly original"
    print("-" * 56)
    print("Verdict            :", verdict)

if __name__ == "__main__":
    sub = sys.argv[1]
    refs = {os.path.splitext(f)[0]: preprocess(load_document(os.path.join(sys.argv[2],
f)))
            for f in sorted(os.listdir(sys.argv[2])) if f.endswith(".txt")}
    words = preprocess(load_document(sub))
    idx, pidx = build_index(refs), build_phrase_index(refs)
    matched, per_src = match_words(words, idx)
    report(os.path.basename(sub), words, matched, per_src, match_phrases(words, pidx))
```

7. RESULTS & DISCUSSION

The program was tested with three reference documents (source1, source2, source3) and two submitted documents. The command used and the output screens obtained are shown below.

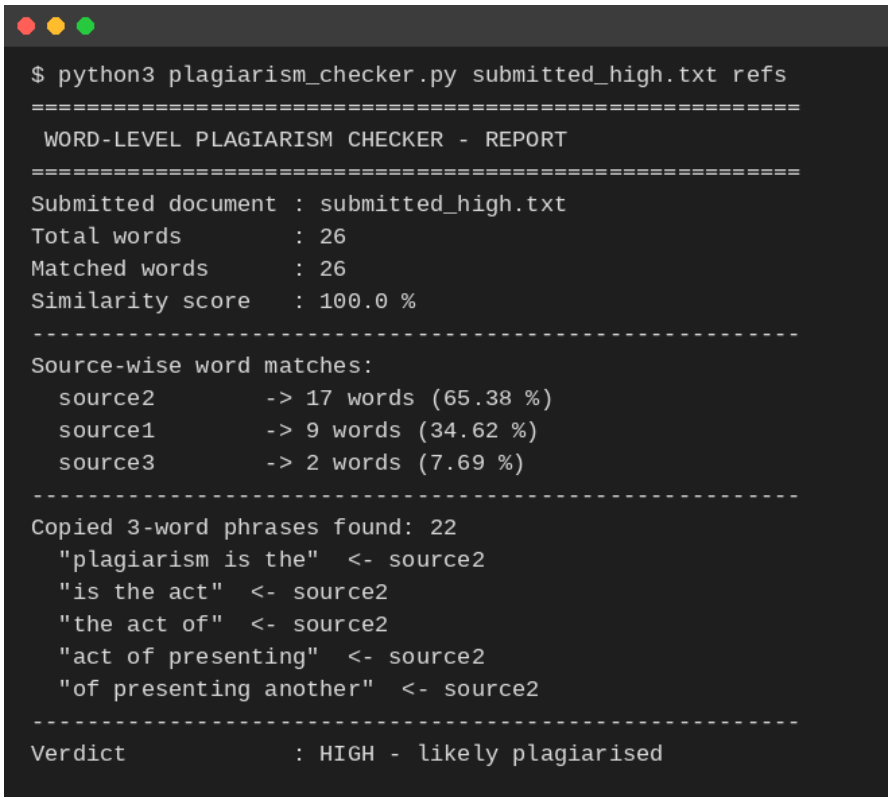


Fig. 1: Output for a submission copied from the reference documents

The submitted text is almost entirely taken from source2 and source1, so all 26 words match and the similarity is 100%. The report lists the contribution of each source and the copied three-word phrases, and gives a HIGH verdict.

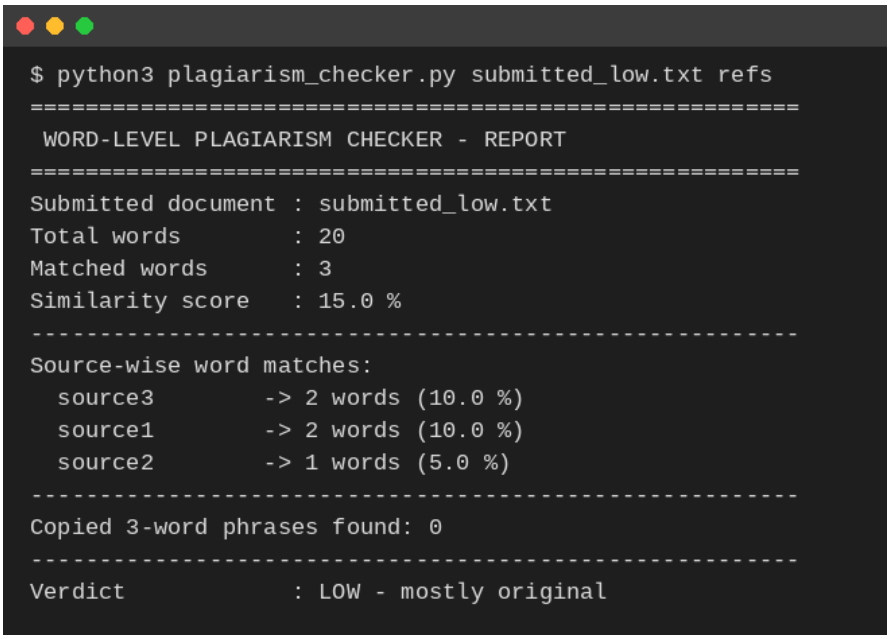


Fig. 2: Output for an original submission

The second submission shares only common words such as "and" and "with" with the references, giving 15% similarity, no copied phrases and a LOW verdict.

Summary of test results

Submitted document	Total words	Matched words	Similarity	Verdict
submitted_high.txt	26	26	100.0 %	HIGH – likely plagiarised
submitted_low.txt	20	3	15.0 %	LOW – mostly original

Discussion: hashing makes each word lookup take O(1) time on average, so the total checking time grows linearly with the number of words in the submission and the reference set, which is suitable for short documents. Because the method is exact word matching, a paraphrased passage with different words will not be detected; this limitation is addressed in the future scope.

8. CONCLUSION & FUTURE SCOPE

Conclusion

The Word-Level Plagiarism Checker was designed and implemented using core data structures and algorithms: arrays and strings for tokenizing the text, a hash table for indexing the reference words, and linear and hash-based searching for matching. All the objectives were met: the system loads the documents, cleans them, finds the matching words and phrases, computes the similarity percentage and reports the source of the matched content. The test runs showed a clear difference between a copied submission (100%) and an original one (15%), so the tool is a quick and lightweight first-level screening aid for teachers, editors and publishers.

Future Scope

- **Near term:** remove stop-words, add stemming and an adjustable n-gram size, and highlight the matched text in the submitted document.
- **Medium term:** support PDF and DOCX input, a graphical or web interface, and a larger reference corpus stored in files or a database.
- **Long term:** detect paraphrased content using semantic similarity (word embeddings), and support multiple languages and online source checking.

9. REFERENCES

[1] Turnitin, “Plagiarism detection software for education,” https://www.turnitin.com

[2] A. Aiken, “MOSS: A System for Detecting Software Similarity,” Stanford University, https://theory.stanford.edu/~aiken/moss/

[3] S. Schleimer, D. S. Wilkerson and A. Aiken, “Winnowing: Local Algorithms for Document Fingerprinting,” ACM SIGMOD, 2003, https://theory.stanford.edu/~aiken/publications/papers/sigmod03.pdf

[4] Wikipedia, “Plagiarism detection,” https://en.wikipedia.org/wiki/Plagiarism_detection

[5] Wikipedia, “tf–idf,” https://en.wikipedia.org/wiki/Tf%E2%80%93idf

[6] Wikipedia, “n-gram,” https://en.wikipedia.org/wiki/N-gram

[7] Wikipedia, “Rabin–Karp algorithm,” https://en.wikipedia.org/wiki/Rabin%E2%80%93Karp_algorithm

[8] GeeksforGeeks, “Hashing Data Structure,” https://www.geeksforgeeks.org/hashing-data-structure/

[9] Python Software Foundation, “Python 3 documentation: dict and re modules,” https://docs.python.org/3/